

A Safe and Modular AGI Architecture

Joachim De Witte

April 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY–NC–ND 4.0 license.

Abstract

This paper presents a safe, modular, and scalable architecture for artificial general intelligence that does not rely on monolithic language models or end-to-end learned systems. Instead, it proposes a structural approach in which general intelligence emerges from the interaction of many small, domain-bound cognitive units coordinated by a lightweight mediator. The core reasoning engine, *Paged-PRE-CAS*, implements explicit recursive processing, epistemic evaluation, and domain isolation through a combination of propose–refine–evaluate cycles, cognitive vector layers, and deterministic paging.

Domain-bound AGI modules encapsulate all domain logic behind three stable interfaces (Domain, State, Action) and operate using externalized search engines rather than parametric learning. These modules may form a fixed library of well-defined domains or, when governance allows, be extended through controlled non-parametric code generation. The architecture is therefore model-agnostic, LLM-agnostic, and compatible with strict safety and traceability requirements.

A mediator-LLM, when present, serves only as a semantic interface for interpretation, translation, and optional domain synthesis; it is not a cognitive core. Reasoning, planning, arbitration, and memory are implemented structurally, not statistically. The full system is designed as a cloud-native cognitive fabric composed of microservices for reasoning, memory, search, evaluators, and experts, enabling traceability, replaceability, and fine-grained governance.

The result is a practical and implementable path toward safe AGI: modular, transparent, pluralistic, and governed by explicit structure rather than emergent behavior. This architecture demonstrates that general intelligence can arise from compositional design and controlled recursion, offering a structural alternative to monolithic scaling and agent-based orchestration.

Contents

1	Introduction	6
2	Motivation	7
2.1	Why This Paper Exists	7
2.2	The Need for a Governable AGI Architecture	7
2.3	Failure Modes of Monolithic LLM Scaling	8
2.4	The Gap Between Agent Frameworks and True Reasoning	8
2.5	Why AGI Must Be Modular and Domain-Bound	8
2.6	LLMs as Optional Semantic Engines	9

3	Related Work	9
3.1	Society of Mind and Agentic Cognition	9
3.2	Cognitive Architectures	9
3.3	Multi-Agent and Tool-Augmented LLM Systems	9
3.4	Modular and Mixture-of-Experts Models	10
3.5	AI Safety Frameworks	10
3.6	Cloud-Native AI and Distributed Cognition	10
3.7	How Paged-PRE-CAS Differs Fundamentally	10
4	Architectural Requirements for Safe AGI	11
4.1	Traceability and Inspectability	11
4.2	Domain Isolation and Bounded Action Spaces	11
4.3	Plurality and Epistemic Diversity	11
4.4	Controlled Recursion	12
4.5	Replaceability and Modular Growth	12
4.6	Governance as an Architectural Property	12
4.7	Why Self-Expansion Must Be Optional and Controlled	13
5	PRE: The Minimal Recursive Reasoning Kernel	13
5.1	The Propose–Refine–Evaluate Cycle	13
5.2	Explicit Internal Communication	13
5.3	Bounded Recursion	14
5.4	Transparency of Intermediate States	14
5.5	PRE as Atomic Reasoning Unit	14
6	CAS: The Cognitive Vector Layer	14
6.1	Epistemic Vectors	15
6.2	Orthogonal Evaluation Dimensions	15
6.3	CAS-Guided Generation and Refinement	15
6.4	CAS-N: Plural Evaluators and Consensus	16
6.5	Emergent Abort Through Disagreement	16
7	Paging: Domain Decomposition of the Action Space	16
7.1	Why Global Action Spaces Are Unsafe	16
7.2	Pages as Domain-Coherent Cognitive Regions	17
7.3	Deterministic Routing	17
7.4	Domain-Specific Experts	17
7.5	Structural Prevention of Domain Errors	18
8	Paged-PRE-CAS: The Full Reasoning Architecture	18
8.1	Step 1 — Paging	18
8.2	Step 2 — PRE-CAS Within a Page	18
8.3	Step 3 — Filtering	19
8.4	Step 4 — Arbiter	19
8.5	Step 5 — Ensembles and Plural Systems	19
8.6	What the Full Architecture Provides	19
9	Domain-Bound AGI: Atomic Cognitive Units	20
9.1	Why AGI Must Be Composed of Domain Intelligences	20
9.2	The Three Core Interfaces	20
9.3	Experts as Autonomous Cognitive Subsystems	21
9.4	Externalized Search	21

9.5	Procedural Memory vs. Parametric Memory	21
9.6	Fixed Domain Libraries vs. Optional Self-Expansion	22
9.7	Why LLMs Are Optional and Replaceable	22
10	Mediator-LLM: Coordination Across Domains	22
10.1	Interpretation and Translation	22
10.2	Routing to Domain-Bound AGIs	23
10.3	Integration of Partial Results	23
10.4	Maintaining Continuity Across Tasks	23
10.5	LLM as Semantic Engine, Not Cognitive Core	23
10.6	Why the Architecture Remains LLM-Agnostic	24
11	Cloud-Native AGI	24
11.1	AGI as a Distributed Cognitive Fabric	24
11.2	Stateless Reasoning Core	24
11.3	Persistent Object Memory	25
11.4	Asynchronous Orchestration	25
11.5	Microservices for Reasoning, Memory, Search, Evaluators, and Experts	26
11.6	Governance Through Observability	26
12	Comparison with Agent Architectures	26
12.1	Why Agents Simulate Reasoning	26
12.2	Why Paged-PRE-CAS Implements Reasoning	27
12.3	Structured State vs. Transcript Accumulation	27
12.4	Real Search vs. Hallucinated Search	27
12.5	Governance and Safety Differences	28
12.6	Complementarity, Not Equivalence	28
13	Real-World Example: Discrete State-Transition Domains	29
13.1	Problem Class Definition	29
13.2	Domain, State, and Action Interfaces	29
13.3	Reasoning Flow via Mediator-LLM	30
13.4	Simulation and Next-Move Prediction	30
13.5	Automatic Domain Creation from Natural Language (Optional)	30
13.6	Deterministic Execution via Script Engine	31
13.7	Empirical Demonstration (GitHub Reference)	31
14	Paged-PRE-CAS as a Generative Reasoning Model	31
14.1	Generative Output: Cognitive Programs	31
14.2	Generative Process: PRE-CAS Recursion	32
14.3	Generative Model Without Parametric Learning	32
14.4	Comparison with Token-Based Generative Models	33
14.5	Generative Diversity Through Plural Evaluators	33
14.6	Execution of Generated Programs	33
14.7	Why This Constitutes a Generative Reasoning Model	34
15	Safety Analysis	34
15.1	Structural vs. Behavioural Safety	34
15.2	Domain Isolation as a Primary Safety Mechanism	34
15.3	Bounded Recursion and Controlled Reasoning Depth	35
15.4	Plural Evaluators and Epistemic Diversity	35
15.5	Deterministic Arbitration	35

15.6	Script Engine Execution Guarantees	36
15.7	Governance Through Observability	36
15.8	Comparison with Monolithic LLM Safety	36
15.9	Failure Modes and Mitigations	36
15.10	Summary of Safety Guarantees	37
16	Discussion	37
16.1	A Shift from Parametric to Structural Intelligence	37
16.2	Implications for AGI Safety	38
16.3	Implications for AGI Capability	38
16.4	Novelty of the Architecture	38
16.5	Relationship to Existing Paradigms	39
16.6	Broader Implications	39
17	Limitations and Open Questions	39
17.1	Engineering Complexity and System Integration	39
17.2	Scaling Domain Libraries	40
17.3	Evaluator Design and Epistemic Geometry	40
17.4	Search Engine Selection and Optimization	40
17.5	Mediator-LLM Reliability	40
17.6	Emergent Behaviour in Large-Scale Systems	41
17.7	Human–AI Interaction and Usability	41
17.8	Theoretical Foundations	41
18	Testable Predictions	41
18.1	Prediction 1: CAS-N Disagreement Correlates with Epistemic Uncertainty	41
18.2	Prediction 2: Domain Isolation Eliminates Cross-Domain Error Propagation . . .	42
18.3	Prediction 3: PRE Recursion Depth Exhibits Diminishing Returns	42
18.4	Prediction 4: Externalized Search Outperforms LLM-Simulated Search	42
18.5	Prediction 5: Arbitration Controls, Rather Than Eliminates, Variance	42
18.6	Prediction 6: Ensembles of Reasoning Cores Improve Robustness	43
18.7	Prediction 7: Script Engine Execution Eliminates Hallucination	43
18.8	Prediction 8: Mediator-LLM Errors Are Contained and Non-Catastrophic	43
18.9	Prediction 9: Domain-Bound AGIs Scale Linearly with Domain Count	43
18.10	Prediction 10: Observability Enables Predictable Failure Modes	44
19	Conclusion	44
A	PRE–CAS Overview Table	44
B	Empirical Demonstration of Paged-PRE-CAS as a Generative Reasoning Model	45
B.1	Micro-Domain Setup	46
B.2	Generative Reasoning Loop	46
C	Ablation Studies	46
C.1	Effect of Refinement	46
C.2	Effect of Evaluators	47
C.2.1	Coherence Evaluator Disabled	47
C.2.2	Style Evaluator Disabled	47
C.2.3	Heuristic Structural Evaluators Disabled	47
C.3	Effect of Arbiter Threshold	47

D	Summary of Findings	47
E	Implications for the Main Architecture	48

1 Introduction

Artificial intelligence has advanced rapidly through the scaling of large neural models, yet these systems remain structurally limited. Monolithic architectures entangle all cognitive functions within a single parameter space, making them opaque, difficult to govern, and inherently unsafe for general-purpose reasoning. Their behaviour is shaped by statistical correlations rather than explicit cognitive structure, and their internal processes cannot be decomposed, inspected, or constrained. As a result, scaling alone cannot yield a safe or general intelligence.

This paper proposes a fundamentally different approach: a modular, transparent, and governable architecture in which general intelligence emerges from the interaction of many small, domain-bound cognitive units. Rather than relying on a single model to internalize all knowledge and reasoning strategies, the architecture decomposes cognition into explicit roles, explicit interfaces, and explicit control flows. Reasoning is implemented structurally through recursive refinement, epistemic evaluation, and domain isolation, not through emergent behaviour in a monolithic network.

The design draws inspiration from Minsky’s *Society of Mind*, which argued that intelligence arises from the coordinated activity of many simple processes. Modern developments in Hybrid Intelligence—systems in which humans and machines jointly participate in recursive reasoning loops—further demonstrate that cognition is most stable and most general when distributed across heterogeneous components. Building on these insights, the architecture introduced here formalizes a compositional model of intelligence suitable for real-world deployment.

At the core of the system lies *Paged-PRE-CAS*, a generative reasoning engine that performs explicit propose–refine–evaluate cycles under the guidance of cognitive vector layers (CAS) and domain-specific paging. This engine does not rely on language models for reasoning; instead, it synthesizes cognitive programs that are executed deterministically by a script engine. Domain logic is encapsulated in modular, replaceable components called *domain-bound AGI modules*, each implementing a stable interface for states, actions, and heuristics. Search is externalized and pluggable, enabling deep reasoning without parametric learning.

Language models, when present, serve only as semantic mediators: translating natural language into structured representations, generating optional domain interfaces, or providing heuristic suggestions. They are not cognitive cores, and the architecture remains fully operational without them. This separation ensures that reasoning, planning, arbitration, and memory remain transparent, auditable, and structurally constrained.

The resulting system is inherently safe. Domain isolation prevents cross-domain errors, bounded recursion ensures stability, plural evaluators provide epistemic diversity, and deterministic execution enables full traceability. The architecture is cloud-native by design, decomposing reasoning, memory, search, evaluators, and experts into microservices that can be governed, monitored, and replaced independently.

Contributions of This Paper

This paper makes four primary contributions:

- (i) It introduces a modular and governable AGI architecture based on explicit cognitive structure rather than monolithic statistical models.
- (ii) It formalizes *Paged-PRE-CAS* as a generative reasoning engine capable of controlled recursion, epistemic evaluation, and domain isolation.
- (iii) It defines *domain-bound AGI modules* as atomic cognitive units that encapsulate domain logic behind stable interfaces and operate using externalized search.
- (iv) It presents a cloud-native execution model in which reasoning, memory, search, and evaluation are implemented as distributed microservices.

Together, these contributions outline a practical and implementable path toward safe AGI: modular, transparent, pluralistic, and grounded in explicit structure rather than emergent behaviour.

2 Motivation

The rapid progress of large neural models has created the impression that general intelligence may emerge from continued scaling of monolithic architectures. However, these systems exhibit structural limitations that make them unsuitable as foundations for safe and governable AGI. Their internal representations are deeply entangled, their reasoning processes are opaque, and their behaviour cannot be decomposed into stable cognitive roles. As a result, they lack the modularity, traceability, and controllability required for long-horizon reasoning or high-stakes decision-making.

This paper is motivated by the need for an alternative path: an AGI architecture grounded in explicit structure rather than emergent statistical behaviour. The goal is not to produce a larger or more capable model, but to design a system whose cognitive organization is transparent, decomposable, and inherently safe. The architecture presented here is built on three principles: modularity, explicit reasoning, and domain isolation. These principles ensure that cognitive processes remain inspectable, replaceable, and bounded, even as the system grows in capability.

2.1 Why This Paper Exists

This work is part of the broader Triadic Cosmos ecosystem, which aims to articulate minimal structural principles underlying both physical systems and intelligent behaviour. While the companion work *The Triadic Cycle as Unified Framework* develops a physical ontology, the present paper focuses on the computational organization of intelligence. Its purpose is to provide a practical, implementable architecture that embodies the structural principles identified in the ecosystem: multiplicity, recursion, representational richness, and energetic grounding.

The architecture is not intended as a theoretical abstraction but as a blueprint for real systems. It is designed to be implementable with current technology, compatible with cloud-native infrastructure, and suitable for open-source development. The motivation is therefore both conceptual and practical: to provide a coherent alternative to monolithic AI and to demonstrate how safe AGI can be constructed from modular components.

2.2 The Need for a Governable AGI Architecture

Safety in AGI cannot be achieved through alignment patches or post-hoc interpretability tools. It must be embedded in the architecture itself. A safe system must satisfy several structural requirements:

- **Traceability:** every cognitive step must be inspectable and reproducible.
- **Domain isolation:** errors must remain local and must not propagate across domains.
- **Plurality:** multiple evaluators must provide epistemic diversity and prevent collapse.
- **Bounded recursion:** reasoning depth must be controlled and auditable.
- **Replaceability:** components must be modular and independently upgradable.

Monolithic models satisfy none of these requirements. Their internal dynamics are inaccessible, their reasoning is implicit, and their behaviour cannot be decomposed into stable subsystems. A governable AGI must therefore be modular by design.

2.3 Failure Modes of Monolithic LLM Scaling

Large language models have demonstrated impressive capabilities, but they suffer from structural failure modes that make them unsuitable as AGI cores:

- **Entanglement:** all knowledge and reasoning strategies are fused into a single parameter space.
- **Opacity:** internal representations cannot be inspected or constrained.
- **Unbounded action spaces:** models can generate arbitrary outputs without structural safeguards.
- **Lack of domain boundaries:** errors in one domain can influence behaviour in unrelated domains.
- **Transcript-based state:** agents built on LLMs accumulate state in prompts rather than structured memory.

Scaling improves correctness but worsens entanglement, reducing epistemic diversity and increasing the risk of attractor collapse. These limitations are architectural, not quantitative, and cannot be resolved by further scaling.

2.4 The Gap Between Agent Frameworks and True Reasoning

Agent frameworks attempt to simulate reasoning by orchestrating LLM calls, tool invocations, and memory stores. While useful for practical automation, they do not implement genuine cognitive structure. Their reasoning is emergent rather than explicit, their state is stored in unstructured transcripts, and their planning relies on prompting rather than algorithmic search. As a result, they remain fragile, unpredictable, and unsuitable as foundations for AGI.

A true reasoning architecture must implement explicit control flow, explicit memory, explicit evaluation, and explicit recursion. It must be able to decompose tasks, refine hypotheses, and integrate results in a structured manner. This paper introduces such an architecture.

2.5 Why AGI Must Be Modular and Domain-Bound

General intelligence does not require a single model that internalizes all domains. Instead, it can emerge from the interaction of many small, domain-bound intelligences, each with its own representational geometry, heuristics, and constraints. Domain-bound AGI modules provide:

- **Local coherence:** each module is internally consistent and domain-specific.
- **Replaceability:** modules can be upgraded or swapped without destabilizing the system.
- **Safety:** domain boundaries prevent cross-domain errors.
- **Scalability:** new domains can be added without retraining the entire system.

Self-expansion through code generation is optional and governance-sensitive. A system may operate safely with a fixed library of domains, avoiding the risks associated with uncontrolled growth.

2.6 LLMs as Optional Semantic Engines

In this architecture, language models are not cognitive cores. They serve only as semantic interfaces:

- translating natural language into structured representations,
- generating optional domain interfaces from formal descriptions,
- providing heuristics or summaries when appropriate.

Reasoning, planning, arbitration, and memory are implemented structurally, not statistically. The architecture remains fully operational without LLMs, ensuring that its core cognitive processes remain transparent, deterministic, and governable.

3 Related Work

Research on cognitive architectures, modular AI systems, and distributed reasoning spans several decades. While these lines of work provide valuable insights, none of them achieve the structural properties required for safe and governable AGI. This section situates the proposed architecture within the broader landscape of cognitive science, artificial intelligence, and systems engineering, highlighting both conceptual connections and fundamental differences.

3.1 Society of Mind and Agentic Cognition

Minsky’s *Society of Mind* Minsky 1986 proposed that intelligence emerges from the interaction of many small, specialized agents rather than from a single monolithic process. This perspective anticipated modern views of distributed cognition and modular intelligence.

However, the agents in Society of Mind lack explicit interfaces, epistemic evaluators, domain boundaries, and deterministic control flow. The framework does not specify how agents should coordinate, how conflicts should be resolved, or how reasoning should be governed at scale.

The architecture presented in this paper can be seen as a structural and computational realization of the Society of Mind intuition: domain-bound AGIs, evaluators, search engines, and reasoning cores act as explicit, inspectable components whose interactions are governed by PRE recursion, CAS evaluation, and deterministic arbitration. Rather than emergent coordination, the system provides formal protocols for cognitive interaction.

3.2 Cognitive Architectures

Classical cognitive architectures such as Soar Laird 2012, ACT-R Anderson 2007, CLARION Sun 2016, and Global Workspace Theory Baars 1988; Dehaene, Kerszberg, and Changeux 1998 model cognition as structured interactions between symbolic components, memory systems, and control processes. These frameworks emphasize modularity, explicit reasoning, and interpretable internal state.

However, they rely on hand-crafted rules, lack scalable learning mechanisms, and do not provide a path toward domain expansion or cloud-native execution. The architecture presented in this paper inherits their emphasis on structure and transparency while introducing scalable recursion, plural evaluators, and domain-bound modularity.

3.3 Multi-Agent and Tool-Augmented LLM Systems

Recent frameworks such as AutoGen Wu et al. 2023, LangChain Chase 2023, and Microsoft’s Semantic Kernel Microsoft 2023 orchestrate large language models with external tools, memory

stores, and multi-agent workflows. These systems demonstrate practical utility but do not implement genuine cognitive structure.

Their reasoning is emergent rather than explicit, their state is stored in unstructured transcripts, and their planning relies on prompting rather than algorithmic control. They lack domain isolation, deterministic execution, and epistemic governance. In contrast, the architecture proposed here implements reasoning as a first-class computational process, independent of language models.

3.4 Modular and Mixture-of-Experts Models

Mixture-of-experts architectures Shazeer et al. 2017; Lepikhin et al. 2021; Fedus, Zoph, and Shazeer 2022 introduce modularity at the level of neural computation, enabling specialization and improved efficiency. While these models demonstrate the benefits of distributed computation, they remain monolithic in their parameterization and do not provide explicit cognitive roles, domain boundaries, or traceable reasoning steps.

Their modularity is statistical rather than structural. The present architecture extends modularity to the cognitive level, defining explicit interfaces for domain logic, reasoning, evaluation, and memory.

3.5 AI Safety Frameworks

Work on interpretability Olah et al. 2020; Nanda et al. 2023, alignment Christiano et al. 2017; Ouyang et al. 2022, and governance Amodei et al. 2016; Hendrycks et al. 2022 highlights the need for transparency, controllability, and bounded behaviour in advanced AI systems. However, most safety frameworks assume monolithic models and attempt to impose safety constraints post hoc.

The architecture introduced here embeds safety into the structure itself: domain isolation, bounded recursion, plural evaluators, and deterministic execution provide inherent safeguards that do not depend on behavioural heuristics or alignment patches.

3.6 Cloud-Native AI and Distributed Cognition

Distributed AI systems Dean and Ghemawat 2004; Moritz et al. 2018, microservice architectures Newman 2015, and cloud-native orchestration frameworks Authors 2023 demonstrate how large-scale computation can be decomposed into independent, replaceable components. Similarly, research on collective intelligence and distributed cognition Hutchins 1995; Woolley et al. 2010 shows that reasoning can emerge from the interaction of multiple agents.

The proposed architecture synthesizes these ideas into a unified cognitive fabric: reasoning, memory, search, evaluators, and experts operate as microservices coordinated through explicit protocols.

3.7 How Paged-PRE-CAS Differs Fundamentally

While related work provides important conceptual foundations, the architecture presented in this paper differs in three fundamental ways:

- (i) **Reasoning is explicit:** Paged-PRE-CAS synthesizes cognitive programs rather than relying on emergent behaviour.
- (ii) **Modularity is structural:** domain-bound AGI modules encapsulate domain logic behind stable interfaces.
- (iii) **Safety is architectural:** domain isolation, plural evaluators, and deterministic execution are built into the system.

These differences position the architecture as a structural alternative to both monolithic scaling and agent-based orchestration.

4 Architectural Requirements for Safe AGI

A safe AGI system cannot rely on behavioural heuristics, alignment patches, or post-hoc interpretability tools. Safety must be embedded in the architecture itself. This section identifies the structural requirements that any AGI system must satisfy in order to remain governable, transparent, and stable under recursive reasoning. These requirements motivate the design choices underlying the Paged-PRE-CAS architecture and the domain-bound AGI model.

4.1 Traceability and Inspectability

Every cognitive step must be observable, reproducible, and auditable. A safe AGI architecture must expose:

- intermediate reasoning states,
- evaluator scores and epistemic vectors,
- search decisions and branching factors,
- memory reads and writes,
- arbitration outcomes.

Monolithic models do not satisfy this requirement: their internal activations are opaque, their reasoning is implicit, and their outputs cannot be decomposed into meaningful steps. A safe architecture must therefore implement explicit reasoning and deterministic execution.

4.2 Domain Isolation and Bounded Action Spaces

General-purpose models operate over unbounded action spaces, allowing arbitrary outputs that may violate domain constraints. A safe AGI must enforce:

- strict domain boundaries,
- domain-specific action vocabularies,
- deterministic routing to domain experts,
- structural prevention of cross-domain errors.

Domain isolation ensures that errors remain local and that reasoning remains coherent within each domain. It also enables fine-grained governance and replaceability of domain modules.

4.3 Plurality and Epistemic Diversity

A single evaluator or reasoning pathway is insufficient for robust cognition. Safe AGI requires epistemic plurality:

- multiple evaluators with independent error geometries,
- consensus mechanisms for stability,
- disagreement-triggered aborts for safety,

- orthogonal evaluation dimensions (safety, accuracy, relevance, domain coherence).

Plurality prevents attractor collapse and provides natural uncertainty detection. It also ensures that no single subsystem becomes a single point of failure.

4.4 Controlled Recursion

Recursive reasoning is essential for planning, abstraction, and self-correction, but unbounded recursion is unsafe. A safe architecture must enforce:

- bounded recursion depth,
- explicit recursion steps,
- evaluator-guided refinement,
- deterministic termination conditions.

These constraints ensure that reasoning remains stable and predictable, even in long-horizon tasks.

4.5 Replaceability and Modular Growth

A safe AGI must be able to evolve without destabilizing its existing capabilities. This requires:

- modular components with stable interfaces,
- domain-bound AGI modules that can be added or replaced independently,
- externalized search engines that can be swapped without retraining,
- a reasoning core that contains no domain-specific knowledge.

Replaceability ensures that the system can grow structurally rather than parametrically, avoiding the risks associated with weight updates in monolithic models.

4.6 Governance as an Architectural Property

Governance must be embedded into the architecture, not added externally. This includes:

- audit logs for all cognitive operations,
- deterministic execution paths,
- explicit control flow,
- policy layers that constrain domain access and recursion depth,
- cloud-native observability (metrics, traces, structured logs).

A governable AGI is one whose behaviour can be monitored, constrained, and verified at every step.

4.7 Why Self-Expansion Must Be Optional and Controlled

While the architecture supports the automatic creation of new domain modules through non-parametric code generation, this capability is optional and governance-sensitive. A safe AGI may operate with a fixed library of domains, avoiding the risks associated with uncontrolled growth. When self-expansion is enabled, it must be:

- explicitly authorized,
- fully traceable,
- sandboxed,
- subject to evaluator scrutiny,
- reversible and replaceable.

This ensures that the system remains stable and predictable even as it acquires new capabilities.

5 PRE: The Minimal Recursive Reasoning Kernel

At the foundation of the architecture lies a minimal but fully expressive reasoning mechanism: the *Propose–Refine–Evaluate* (PRE) kernel. PRE provides the smallest structural unit in which reasoning becomes explicit, traceable, and recursively improvable. Unlike monolithic models, which internalize reasoning implicitly within a single parameter space, PRE decomposes reasoning into three explicit cognitive roles with transparent communication channels.

5.1 The Propose–Refine–Evaluate Cycle

The PRE kernel operates through a controlled recursive loop:

- (i) **Propose**: generate an initial hypothesis or candidate solution.
- (ii) **Evaluate**: assess the candidate using explicit evaluators.
- (iii) **Refine**: improve the candidate based on evaluator feedback.

This cycle may repeat for a bounded number of iterations, producing increasingly coherent and domain-aligned outputs. Each step is explicit, inspectable, and recorded in the system’s structured memory, enabling full traceability of the reasoning process.

5.2 Explicit Internal Communication

A defining feature of PRE is its use of explicit messages between roles. The proposer does not implicitly “learn” evaluator preferences; instead, evaluators communicate structured feedback that guides refinement. This separation ensures:

- transparency of intermediate reasoning states,
- modularity of evaluators,
- replaceability of refinement strategies,
- clear cognitive boundaries between roles.

Explicit communication prevents entanglement and enables fine-grained governance over the reasoning process.

5.3 Bounded Recursion

Recursive reasoning is powerful but potentially unstable. PRE enforces bounded recursion through:

- fixed recursion depth limits,
- evaluator-driven termination conditions,
- structural aborts triggered by evaluator disagreement,
- deterministic control flow.

These constraints ensure that reasoning remains predictable and auditable, even in complex multi-step tasks.

5.4 Transparency of Intermediate States

Every PRE iteration produces:

- a candidate hypothesis,
- evaluator scores across orthogonal dimensions,
- refinement instructions,
- a traceable record of changes.

This transparency enables debugging, governance, and reproducibility. It also allows external systems—human or machine—to inspect and intervene in the reasoning process when necessary.

5.5 PRE as Atomic Reasoning Unit

PRE is intentionally minimal. It does not assume:

- any specific model architecture,
- any particular search algorithm,
- any domain-specific knowledge,
- any reliance on language models.

Instead, PRE provides a substrate-independent reasoning primitive that can be composed, parallelized, or embedded within larger cognitive structures. In the full architecture, PRE serves as the atomic reasoning unit inside each page of the Paged-PRE-CAS system, enabling controlled recursion, epistemic evaluation, and modular refinement.

6 CAS: The Cognitive Vector Layer

While PRE provides the minimal recursive mechanism for generating and refining hypotheses, it does not specify *how* candidates should be evaluated or *which* cognitive dimensions matter for a given task. The *Cognitive Vector Layer* (CAS) fills this gap by attaching an epistemic profile to every intermediate and final output. CAS transforms reasoning from an unstructured refinement loop into a guided, multi-dimensional cognitive process.

6.1 Epistemic Vectors

A CAS vector is a structured representation of the epistemic properties of a candidate. It encodes orthogonal evaluation dimensions such as:

- **safety**: adherence to domain constraints and system policies,
- **accuracy**: correctness relative to domain heuristics or ground truth,
- **relevance**: alignment with the task or query,
- **style**: formatting, tone, or structural coherence,
- **domain**: compatibility with the active domain or page.

These dimensions are not tied to any specific model or evaluator. Instead, they form a general epistemic geometry that can be implemented by heterogeneous evaluators, including symbolic checkers, search heuristics, or neural scoring modules.

6.2 Orthogonal Evaluation Dimensions

A key design principle of CAS is the separation of evaluation dimensions. Safety, accuracy, relevance, and domain coherence are evaluated independently, preventing entanglement and enabling fine-grained control. This orthogonality provides several benefits:

- **transparent evaluation**: each dimension can be inspected separately,
- **modular governance**: policy layers can target specific dimensions,
- **robustness**: failure in one dimension does not corrupt others,
- **composability**: evaluators can be added or replaced independently.

Orthogonality ensures that reasoning remains interpretable and that evaluators do not interfere with one another.

6.3 CAS-Guided Generation and Refinement

During the PRE cycle, CAS vectors guide both refinement and arbitration:

- the **proposer** generates candidates conditioned on a target CAS vector,
- the **evaluators** compute CAS scores for each candidate,
- the **refiner** adjusts the candidate to better match the target vector.

This creates a closed-loop control system in which reasoning is steered by explicit epistemic goals. CAS therefore acts as a cognitive control layer, shaping the trajectory of recursive reasoning.

6.4 CAS-N: Plural Evaluators and Consensus

To ensure epistemic diversity and robustness, CAS can be extended to *CAS-N*, a plural evaluation system in which multiple independent evaluators compute CAS vectors using different internal error geometries. CAS-N provides:

- **epistemic diversity**: evaluators trained or designed with different biases,
- **consensus**: stable agreement across evaluators,
- **disagreement-based aborts**: structural safety mechanism,
- **uncertainty detection**: natural signal for ambiguous or unsafe states.

Plurality prevents attractor collapse and ensures that no single evaluator dominates the reasoning process.

6.5 Emergent Abort Through Disagreement

A distinctive property of CAS-N is that disagreement between evaluators triggers an automatic abort. This mechanism is not heuristic but structural: if evaluators with independent error geometries diverge significantly, the system halts the reasoning process and returns an uncertainty state. This provides:

- **built-in safety**: unsafe or incoherent candidates are rejected,
- **robustness**: errors do not propagate through recursion,
- **interpretability**: disagreement signals epistemic instability.

This emergent abort mechanism is a core safety feature of the architecture.

7 Paging: Domain Decomposition of the Action Space

Reasoning systems that operate over a global, undifferentiated action space are inherently unsafe. Without structural boundaries, a system may generate actions that violate domain constraints, mix incompatible representations, or apply inappropriate heuristics. Paging addresses this problem by decomposing the global action space into coherent, domain-specific regions called *pages*. Each page contains its own experts, action vocabularies, evaluators, and constraints, ensuring that reasoning remains structurally aligned with the active domain.

7.1 Why Global Action Spaces Are Unsafe

Monolithic models and agent-based systems typically operate over a single, universal action space. This design introduces several risks:

- **cross-domain interference**: reasoning strategies from one domain may corrupt another,
- **invalid actions**: the system may generate actions that are syntactically valid but semantically impossible,
- **unbounded behaviour**: the system may explore irrelevant or unsafe regions of the action space,
- **lack of governance**: domain-specific policies cannot be enforced reliably.

Paging eliminates these risks by enforcing strict domain boundaries and deterministic routing.

7.2 Pages as Domain-Coherent Cognitive Regions

A page is a self-contained cognitive region associated with a specific domain. Each page contains:

- domain-specific experts,
- a restricted action vocabulary,
- domain-aware evaluators,
- heuristics and constraints,
- a local PRE-CAS reasoning loop.

Pages ensure that reasoning remains coherent within the domain and that all actions are valid by construction.

7.3 Deterministic Routing

Paging introduces a deterministic routing mechanism that maps the current context to the correct domain page. Routing may be based on:

- the structure of the input state,
- domain metadata,
- explicit domain identifiers,
- mediator-LLM interpretation (when present).

Once a page is selected, all subsequent reasoning steps occur strictly within that page until the task is complete or the system transitions to another domain under controlled conditions.

7.4 Domain-Specific Experts

Each page contains a set of experts responsible for generating, refining, or evaluating actions within the domain. Experts may be:

- symbolic algorithms,
- neural modules,
- search heuristics,
- simulators,
- parsers or validators.

Experts communicate with the reasoning core through explicit callbacks, ensuring that domain logic remains externalized and modular.

7.5 Structural Prevention of Domain Errors

Paging provides a structural guarantee that domain errors cannot occur. This is achieved through:

- **restricted action vocabularies:** only domain-valid actions can be generated,
- **domain-specific evaluators:** invalid states are rejected immediately,
- **page-local reasoning:** PRE-CAS operates only within the active domain,
- **explicit transitions:** domain switches require explicit authorization.

These mechanisms ensure that reasoning remains safe, coherent, and aligned with domain constraints.

8 Paged-PRE-CAS: The Full Reasoning Architecture

The Paged-PRE-CAS architecture integrates the three core components introduced in the previous sections—Paging, PRE, and CAS—into a unified reasoning system. Paging provides domain decomposition and structural safety; PRE implements explicit recursive reasoning; and CAS supplies epistemic guidance and evaluator plurality. Together, they form a modular, transparent, and governable reasoning engine suitable for domain-bound AGI.

8.1 Step 1 — Paging

Reasoning begins with the selection of the appropriate domain page. Paging ensures that:

- the action space is restricted to domain-valid operations,
- domain-specific experts and evaluators are activated,
- cross-domain interference is structurally impossible,
- all reasoning occurs within a coherent representational geometry.

Routing may be performed by structural rules, domain metadata, or a mediator-LLM when present. Once a page is selected, the reasoning core operates strictly within that domain.

8.2 Step 2 — PRE-CAS Within a Page

Within the selected page, reasoning proceeds through a PRE-CAS loop:

- (i) **Propose:** domain experts or neural modules generate candidate actions or states.
- (ii) **Evaluate:** CAS evaluators compute epistemic vectors across orthogonal dimensions.
- (iii) **Refine:** candidates are improved based on evaluator feedback.
- (iv) **Re-evaluate:** updated candidates are scored again.

This loop continues for a bounded number of iterations, producing increasingly coherent and domain-aligned candidates. PRE ensures explicit recursion; CAS ensures epistemic guidance; Paging ensures domain isolation.

8.3 Step 3 — Filtering

After the PRE-CAS loop, the system applies filtering mechanisms:

- **hard constraints:** candidates violating domain rules are discarded,
- **soft thresholds:** candidates must meet minimum epistemic quality,
- **policy filters:** governance layers may impose additional restrictions.

Filtering ensures that only safe, valid, and domain-coherent candidates proceed to arbitration.

8.4 Step 4 — Arbiter

The arbiter combines evaluator scores and selects the best candidate. Arbitration may involve:

- weighted aggregation of CAS dimensions,
- consensus across CAS-N evaluators,
- rejection if evaluators disagree significantly,
- optional diversity mechanisms to avoid local minima.

The arbiter provides a deterministic and transparent decision-making step, ensuring that the system’s output is grounded in explicit epistemic criteria.

8.5 Step 5 — Ensembles and Plural Systems

For additional robustness, multiple Paged-PRE-CAS systems may operate in parallel:

- **ensemble reasoning:** multiple reasoning cores explore different trajectories,
- **plural arbitration:** results are compared or merged,
- **population analysis:** diversity across systems signals uncertainty,
- **redundancy:** failures in one system do not compromise the whole.

Ensembles provide structural pluralism, improving stability and safety.

8.6 What the Full Architecture Provides

Paged-PRE-CAS delivers several key properties required for safe AGI:

- **Modularity:** domain logic, evaluators, and experts are independent components.
- **Traceability:** every reasoning step is explicit and inspectable.
- **Orthogonal constraints:** safety, accuracy, relevance, and domain coherence are separated.
- **Plurality:** multiple evaluators and optional ensembles prevent collapse.
- **Bounded recursion:** reasoning depth is controlled and auditable.
- **Scalability:** new domains can be added without retraining the system.
- **Governability:** deterministic execution and domain isolation enable fine-grained control.

Paged-PRE-CAS therefore constitutes the structural core of a safe, modular, and extensible AGI architecture.

9 Domain-Bound AGI: Atomic Cognitive Units

General intelligence does not require a single system that internalizes all domains. Instead, it can emerge from the interaction of many small, coherent, and independently governed cognitive modules. A *domain-bound AGI* is such a module: a fully functional cognitive subsystem that implements reasoning, evaluation, planning, and action within a single domain. Each domain-bound AGI is internally coherent, externally modular, and structurally isolated from other domains, making it the atomic cognitive unit of the proposed architecture.

9.1 Why AGI Must Be Composed of Domain Intelligences

Different domains require different representational geometries, heuristics, and constraints. Attempting to unify all domains within a single model leads to entanglement, loss of traceability, and unsafe generalization. Domain-bound AGIs avoid these problems by providing:

- **local coherence**: each module maintains its own internal logic and constraints,
- **replaceability**: modules can be upgraded or swapped independently,
- **safety**: domain boundaries prevent cross-domain interference,
- **scalability**: new domains can be added without retraining the system.

General intelligence emerges not from a monolithic model but from the structured interaction of many domain intelligences coordinated by a mediator.

9.2 The Three Core Interfaces

Each domain-bound AGI implements three stable interfaces that define its cognitive boundary:

Domain Interface

- semantic description of the domain,
- detection of whether a state belongs to the domain,
- generation of initial or random states,
- bidirectional translation between text and internal representations,
- selection of the appropriate search engine (Monte Carlo, Beam, A*, heuristics).

State Interface

- access to the linked domain,
- identification of the current actor,
- goal detection,
- enumeration of valid actions,
- state transition under a given action,
- heuristic evaluation of the state,
- optional visualization or description.

Action Interface

- structured representation of an action,
- domain-specific parameters,
- optional visualization or description.

These interfaces form the cognitive API between the reasoning core and domain logic.

9.3 Experts as Autonomous Cognitive Subsystems

Within each domain, experts implement the computational logic required for reasoning. Experts may be:

- symbolic algorithms,
- neural modules,
- simulators,
- parsers or validators,
- heuristic evaluators,
- search engines.

Experts communicate with the reasoning core through explicit callbacks, ensuring that domain logic remains externalized and modular.

9.4 Externalized Search

Domain-bound AGIs do not rely on parametric learning for deep reasoning. Instead, they use externalized search engines such as:

- Monte Carlo search for stochastic or strategic domains,
- Beam Search for deterministic planning,
- A* or constraint solvers for structured optimization,
- domain-specific heuristics for rapid evaluation.

Search is pluggable and domain-specific, enabling deep reasoning without modifying the reasoning core.

9.5 Procedural Memory vs. Parametric Memory

Domain-bound AGIs rely on *procedural memory*—explicit algorithms, heuristics, and interfaces—rather than parametric memory encoded in neural weights. This provides:

- **stability**: no catastrophic forgetting,
- **governance**: algorithms can be inspected and audited,
- **replaceability**: components can be updated without retraining,
- **predictability**: behaviour is determined by explicit procedures.

This stands in contrast to monolithic models, where knowledge and reasoning are entangled in weights.

9.6 Fixed Domain Libraries vs. Optional Self-Expansion

A domain-bound AGI system may operate with a fixed library of domains—e.g., one hundred well-defined planning or optimization domains. This configuration is safe, predictable, and fully governable.

The architecture also supports optional self-expansion through non-parametric code generation, where a mediator-LLM synthesizes new domain interfaces from formal descriptions. However, this capability is:

- optional,
- governance-sensitive,
- fully traceable,
- sandboxed,
- reversible.

Self-expansion is therefore a controlled extension mechanism, not a requirement for AGI.

9.7 Why LLMs Are Optional and Replaceable

Language models are not cognitive cores in this architecture. They serve only as:

- semantic translators between natural language and structured states,
- optional generators of domain interfaces,
- heuristic providers when appropriate.

Reasoning, planning, arbitration, and memory are implemented structurally through Paged-PRE-CAS and domain-bound AGIs. The system remains fully operational without LLMs, ensuring that its core cognitive processes remain transparent, deterministic, and governable.

10 Mediator-LLM: Coordination Across Domains

While domain-bound AGI modules implement reasoning, planning, and search within their own domains, a system capable of interacting with the external world must also interpret natural language, translate user queries into structured representations, and coordinate reasoning across multiple domains. These tasks are performed by the *mediator-LLM*. Importantly, the mediator-LLM is not a cognitive core: it does not perform reasoning, planning, or arbitration. Instead, it serves as a semantic interface and coordination layer.

10.1 Interpretation and Translation

The mediator-LLM receives natural language input from the user and performs three functions:

- Interpretation:** identify the domain or domains relevant to the query.
- Translation:** convert natural language into structured internal representations (states, goals, constraints).
- Normalization:** ensure that the resulting representation conforms to the domain’s interface specifications.

This translation step allows the reasoning core to operate entirely on structured data, avoiding the ambiguity and variability inherent in natural language.

10.2 Routing to Domain-Bound AGIs

Once the input has been translated, the mediator-LLM routes the structured representation to the appropriate domain-bound AGI. Routing may involve:

- domain classification,
- disambiguation across multiple candidate domains,
- decomposition of multi-domain tasks,
- sequential or parallel invocation of domain modules.

Routing is deterministic and auditable: the mediator-LLM produces explicit routing meta-data that can be inspected and verified.

10.3 Integration of Partial Results

Complex tasks may require contributions from multiple domain-bound AGIs. The mediator-LLM integrates partial results by:

- merging structured outputs,
- translating intermediate states between domains,
- coordinating multi-step workflows,
- generating natural language summaries for the user.

Integration is semantic rather than cognitive: the mediator-LLM does not reason about the content but merely transforms and composes representations.

10.4 Maintaining Continuity Across Tasks

The mediator-LLM maintains continuity across user interactions by:

- tracking conversational context,
- preserving task metadata,
- reusing structured states across turns,
- ensuring consistent domain routing.

This continuity layer allows the system to support multi-step tasks without embedding state inside the reasoning core.

10.5 LLM as Semantic Engine, Not Cognitive Core

The mediator-LLM is intentionally limited to semantic tasks. It does not:

- perform reasoning or planning,
- evaluate candidates,
- execute search,
- maintain internal cognitive state,
- modify domain logic.

This separation ensures that the reasoning core remains transparent, deterministic, and governable. The mediator-LLM can be replaced, upgraded, or removed entirely without affecting the system’s cognitive integrity.

10.6 Why the Architecture Remains LLM-Agnostic

Because the mediator-LLM is not a cognitive core, the architecture is fully LLM-agnostic:

- any model capable of semantic translation can serve as mediator,
- multiple mediators may be used for different languages or modalities,
- the system remains operational without an LLM,
- reasoning quality does not depend on LLM scale.

This design avoids the safety and governance risks associated with monolithic LLM-based AGI and ensures that the architecture remains modular, replaceable, and future-proof.

11 Cloud-Native AGI

The architecture presented in this paper is designed from the ground up as a distributed, cloud-native cognitive system. Rather than embedding all reasoning, memory, and evaluation within a single model or process, the system decomposes cognition into independent, communicating microservices. This design enables scalability, observability, fault isolation, and fine-grained governance—properties that are essential for safe AGI deployment in real-world environments.

11.1 AGI as a Distributed Cognitive Fabric

In a cloud-native setting, AGI is not a monolithic program but a *cognitive fabric* composed of interacting services. Each service implements a specific cognitive role:

- reasoning cores (Paged-PRE-CAS),
- domain-bound AGI modules,
- search engines,
- evaluators and CAS-N components,
- memory services,
- mediator-LLM interfaces (optional).

These services communicate through explicit protocols, enabling transparency and modularity. The system can scale horizontally by spawning additional reasoning cores or domain modules as needed.

11.2 Stateless Reasoning Core

The reasoning core is intentionally stateless. It receives:

- a structured state,
- a domain identifier,
- evaluator configurations,
- recursion limits,
- policy constraints.

It produces:

- candidate actions,
- CAS vectors,
- reasoning traces,
- arbitration results.

All persistent information is stored externally in dedicated memory services. Statelessness ensures reproducibility, fault tolerance, and the ability to scale reasoning cores elastically.

11.3 Persistent Object Memory

Long-term memory is implemented as a persistent object store that maintains:

- structured states,
- domain objects,
- task metadata,
- evaluator configurations,
- reasoning traces,
- user context.

Memory is versioned, auditable, and queryable. This allows the system to support multi-step tasks, maintain continuity across interactions, and provide full traceability for governance and debugging.

11.4 Asynchronous Orchestration

Cloud-native AGI relies on asynchronous orchestration to coordinate reasoning, memory, and domain modules. Orchestration frameworks handle:

- task scheduling,
- parallel execution of reasoning cores,
- retries and fault recovery,
- timeouts and recursion limits,
- routing between services.

This enables the system to handle complex workflows, long-horizon tasks, and large-scale reasoning without blocking or resource contention.

11.5 Microservices for Reasoning, Memory, Search, Evaluators, and Experts

Each cognitive function is implemented as a microservice:

- **Reasoning service:** executes Paged-PRE-CAS loops.
- **Memory service:** stores structured states and task metadata.
- **Search service:** provides domain-specific search algorithms.
- **Evaluator service:** computes CAS vectors and performs plural evaluation.
- **Expert services:** implement domain logic, heuristics, and simulators.

This decomposition provides fault isolation: failures in one component do not compromise the entire system. It also enables independent deployment, testing, and upgrading of components.

11.6 Governance Through Observability

Cloud-native infrastructure provides built-in observability mechanisms that are essential for AGI governance:

- **structured logs:** record every reasoning step and evaluator score,
- **metrics:** track recursion depth, evaluator disagreement, and domain transitions,
- **traces:** capture end-to-end reasoning workflows,
- **policy layers:** enforce domain access rules and recursion limits.

Observability transforms AGI from an opaque system into an auditable computational process. Governance becomes a first-class architectural property rather than an afterthought.

12 Comparison with Agent Architectures

Agent-based systems have become a dominant paradigm for orchestrating large language models with tools, memory stores, and external APIs. Frameworks such as AutoGen, LangChain, and Swarm demonstrate practical utility for automation and workflow coordination. However, these systems do not constitute cognitive architectures. Their behaviour emerges from prompting patterns, tool invocation heuristics, and transcript accumulation rather than explicit reasoning structures. This section contrasts agent architectures with the Paged-PRE-CAS framework and highlights the fundamental differences in design, safety, and cognitive organization.

12.1 Why Agents Simulate Reasoning

Agents typically rely on a single large language model to perform all cognitive functions: planning, decomposition, evaluation, memory retrieval, and tool selection. Reasoning is simulated through:

- prompt engineering,
- chain-of-thought generation,
- self-reflection loops,
- tool-calling heuristics,

- multi-agent dialogues.

These mechanisms create the appearance of structured reasoning but lack explicit control flow, bounded recursion, or epistemic governance. The reasoning process remains implicit in the model’s internal activations and cannot be decomposed into stable cognitive roles.

12.2 Why Paged-PRE-CAS Implements Reasoning

In contrast, Paged-PRE-CAS implements reasoning as a first-class computational process:

- PRE provides explicit recursive refinement,
- CAS provides orthogonal epistemic evaluation,
- CAS-N provides plural evaluators and disagreement-based aborts,
- Paging enforces domain isolation and action-space constraints,
- arbitration is deterministic and transparent.

Reasoning is not emergent but algorithmic. Every step is inspectable, reproducible, and governable. This structural clarity is essential for safety and long-horizon planning.

12.3 Structured State vs. Transcript Accumulation

Agent systems typically store state in unstructured transcripts. This leads to:

- uncontrolled context growth,
- implicit memory,
- susceptibility to prompt drift,
- difficulty in debugging or auditing,
- lack of stable intermediate representations.

Paged-PRE-CAS uses structured state representations defined by domain-bound AGIs. State is explicit, typed, and validated by domain interfaces. This enables:

- deterministic state transitions,
- domain-specific constraints,
- reproducible reasoning traces,
- clear separation between memory and reasoning.

12.4 Real Search vs. Hallucinated Search

Agents often rely on LLM-generated search trajectories that are not grounded in actual simulation or algorithmic evaluation. This results in:

- hallucinated search trees,
- inconsistent heuristics,
- unreliable planning,

- lack of guarantees about optimality or safety.

Domain-bound AGIs use externalized search engines—Monte Carlo, Beam, A*, constraint solvers—ensuring that search is:

- algorithmic,
- deterministic,
- domain-valid,
- inspectable,
- reproducible.

Reasoning is grounded in actual computation rather than linguistic simulation.

12.5 Governance and Safety Differences

Agent architectures provide limited safety guarantees:

- no domain isolation,
- no bounded recursion,
- no evaluator plurality,
- no deterministic arbitration,
- no structural abort mechanisms.

Paged-PRE-CAS embeds safety into the architecture:

- domain boundaries prevent cross-domain errors,
- recursion depth is explicitly bounded,
- CAS-N provides epistemic diversity,
- arbitration is deterministic,
- evaluator disagreement triggers automatic aborts.

Safety is structural, not behavioural.

12.6 Complementarity, Not Equivalence

Agent systems and Paged-PRE-CAS are not competing paradigms but complementary layers:

- agent frameworks excel at workflow orchestration and tool integration,
- Paged-PRE-CAS provides a principled reasoning substrate,
- domain-bound AGIs provide structured domain intelligence,
- mediator-LLMs provide semantic translation.

Agents can orchestrate Paged-PRE-CAS modules, but they cannot replace them. The architecture presented here provides the cognitive foundation that agent systems lack.

13 Real-World Example: Discrete State-Transition Domains

To illustrate the practical applicability of the architecture, this section presents a concrete example based on discrete state-transition domains. These domains are common in planning, optimization, simulation, and game-like environments. They provide a clean setting in which the interaction between domain-bound AGIs, search engines, and the Paged-PRE-CAS reasoning core can be demonstrated with full transparency.

13.1 Problem Class Definition

A discrete state-transition domain is defined by:

- a finite or countably infinite set of states S ,
- a set of actions $A(s)$ available in each state $s \in S$,
- a deterministic or stochastic transition function $T(s, a)$,
- a goal condition $G(s)$,
- optional heuristics $h(s)$ for search.

This class includes puzzles, board games, logistics problems, scheduling tasks, and many real-world planning scenarios. The structure aligns naturally with the Domain/State/Action interfaces defined earlier.

13.2 Domain, State, and Action Interfaces

A domain-bound AGI for a discrete state-transition domain implements:

Domain Interface

- detection of whether a state belongs to the domain,
- generation of initial or random states,
- translation between text and structured states,
- selection of the appropriate search engine.

State Interface

- enumeration of valid actions,
- transition under a given action,
- goal detection,
- heuristic evaluation,
- visualization or textual description.

Action Interface

- structured representation of an action,
- domain-specific parameters,
- optional visualization.

These interfaces allow the reasoning core to operate independently of domain-specific logic.

13.3 Reasoning Flow via Mediator-LLM

When a user provides a natural language query (e.g., “Solve this puzzle” or “Plan the next move”), the mediator-LLM performs:

- (i) **Interpretation:** identify the relevant domain (e.g., a puzzle domain).
- (ii) **Translation:** convert the input into a structured state.
- (iii) **Routing:** send the state to the appropriate domain-bound AGI.

The mediator-LLM does not perform reasoning; it only prepares the structured input required by the reasoning core.

13.4 Simulation and Next-Move Prediction

Within the domain, the reasoning core executes a Paged-PRE-CAS loop:

- (i) **Propose:** generate candidate next actions using domain experts or search.
- (ii) **Evaluate:** compute CAS vectors for each candidate.
- (iii) **Refine:** improve candidates based on evaluator feedback.
- (iv) **Arbitrate:** select the best candidate.

If search is required, the domain-bound AGI invokes:

- Monte Carlo search for stochastic domains,
- Beam Search for deterministic planning,
- A* for structured optimization,
- domain-specific heuristics for rapid evaluation.

The result is a next-move prediction or a full plan, depending on the task.

13.5 Automatic Domain Creation from Natural Language (Optional)

When governance allows, the mediator-LLM can generate new domain modules from natural language descriptions. For example:

“Create a domain for a 4×4 sliding puzzle with numbered tiles.”

The mediator-LLM produces:

- a Domain interface,
- a State representation,
- an Action vocabulary,
- transition logic,
- heuristics.

This capability is optional and fully traceable. It allows rapid prototyping of new domains without modifying the reasoning core.

13.6 Deterministic Execution via Script Engine

Once a cognitive program is synthesized by the reasoning core, it is executed by a deterministic script engine. This ensures:

- reproducibility,
- auditability,
- safety,
- independence from neural model variability.

The script engine executes only domain-valid actions, providing a final layer of structural safeguards.

13.7 Empirical Demonstration (GitHub Reference)

A full implementation of this architecture for discrete state-transition domains is available in the accompanying open-source repository. It includes:

- multiple domain-bound AGIs (puzzles, planning tasks, simulations),
- a complete Paged-PRE-CAS reasoning core,
- search engines (Monte Carlo, Beam, A*),
- a deterministic script engine,
- examples of mediator-LLM domain synthesis.

This implementation demonstrates that the architecture is not theoretical but practical, deployable, and compatible with real-world systems.

14 Paged-PRE-CAS as a Generative Reasoning Model

Although the architecture is not based on monolithic neural models, it is nonetheless a *generative* system. Instead of generating text or tokens, Paged-PRE-CAS generates *cognitive programs*: structured sequences of actions, refinements, evaluations, and state transitions that solve a task. These programs are explicit, inspectable, and executable by a deterministic script engine. This section formalizes Paged-PRE-CAS as a generative reasoning model and contrasts it with token-based generative models.

14.1 Generative Output: Cognitive Programs

A cognitive program is a structured object composed of:

- a sequence of domain-valid actions,
- intermediate states,
- evaluator feedback,
- refinement steps,
- arbitration decisions,

- termination conditions.

Unlike token sequences, cognitive programs are:

- **typed**: each element has a domain-specific type,
- **validated**: every action is checked by domain interfaces,
- **executable**: programs can be run deterministically,
- **inspectable**: every step is recorded and auditable.

This makes Paged-PRE-CAS a generative model in the computational sense, not the linguistic one.

14.2 Generative Process: PRE-CAS Recursion

The generative process unfolds through recursive PRE-CAS cycles:

- (i) **Propose**: generate candidate cognitive steps or partial programs.
- (ii) **Evaluate**: compute CAS vectors for each candidate.
- (iii) **Refine**: improve candidates based on evaluator feedback.
- (iv) **Arbitrate**: select the best candidate.

Each iteration extends or modifies the cognitive program. The recursion continues until:

- a goal state is reached,
- evaluators converge,
- recursion limits are reached,
- disagreement triggers an abort.

The result is a complete, domain-valid cognitive program.

14.3 Generative Model Without Parametric Learning

Paged-PRE-CAS is generative but not parametric. It does not rely on:

- neural weights,
- gradient descent,
- token prediction,
- statistical correlations.

Instead, generation emerges from:

- explicit search,
- domain heuristics,
- evaluator feedback,
- structural recursion,
- deterministic arbitration.

This makes the system fully transparent and governable.

14.4 Comparison with Token-Based Generative Models

Token-based generative models (LLMs) generate sequences of tokens based on statistical patterns. Their generative process is:

- implicit,
- entangled,
- opaque,
- non-deterministic,
- difficult to govern.

Paged-PRE-CAS differs fundamentally:

- it generates structured cognitive programs, not text,
- its generative process is explicit and algorithmic,
- every step is traceable and auditable,
- domain boundaries constrain generation,
- evaluators provide orthogonal epistemic guidance.

This distinction is central to the architecture’s safety and modularity.

14.5 Generative Diversity Through Plural Evaluators

Plural evaluators (CAS-N) introduce diversity into the generative process:

- different evaluators explore different cognitive trajectories,
- disagreement signals uncertainty or unsafe states,
- consensus ensures stability,
- ensembles of reasoning cores provide additional diversity.

Generative diversity is therefore structural rather than stochastic.

14.6 Execution of Generated Programs

Once a cognitive program is generated, it is executed by a deterministic script engine. This ensures:

- reproducibility across runs,
- independence from neural variability,
- strict adherence to domain constraints,
- full traceability for governance.

Execution is separate from generation, providing an additional layer of safety.

14.7 Why This Constitutes a Generative Reasoning Model

Paged-PRE-CAS satisfies the criteria for a generative model:

- it produces structured outputs,
- its outputs are novel but domain-valid,
- its generative process is recursive and constructive,
- it can generate arbitrarily long cognitive programs,
- it can generalize across tasks within a domain.

However, it does so through explicit reasoning rather than statistical prediction. This makes it a generative reasoning model: a system that generates solutions through structured cognitive processes rather than token-level correlations.

15 Safety Analysis

Safety in AGI cannot be achieved through behavioural heuristics, alignment patches, or post-hoc interpretability tools. It must be embedded directly into the architecture. This section analyzes how the structural components of the Paged-PRE-CAS framework—Paging, PRE, CAS, CAS-N, and domain-bound AGIs—collectively enforce safety guarantees that are unachievable in monolithic or agent-based systems.

15.1 Structural vs. Behavioural Safety

Most contemporary AI safety approaches rely on behavioural methods:

- reinforcement learning from human feedback,
- prompt-based guardrails,
- post-hoc filtering,
- adversarial training,
- interpretability tools applied after training.

These methods attempt to constrain the behaviour of opaque models whose internal representations remain inaccessible. They cannot guarantee safety under distribution shift, recursive reasoning, or long-horizon planning.

In contrast, Paged-PRE-CAS provides *structural safety*: safety emerges from the architecture itself, not from behavioural tuning.

15.2 Domain Isolation as a Primary Safety Mechanism

Paging enforces strict domain boundaries. This prevents:

- cross-domain interference,
- invalid action generation,
- representational drift,
- unsafe generalization,

- domain mixing during recursive reasoning.

Because each domain-bound AGI defines its own action vocabulary, constraints, and evaluators, the system cannot generate actions outside the domain’s permitted space. This eliminates entire classes of unsafe behaviour.

15.3 Bounded Recursion and Controlled Reasoning Depth

Recursive reasoning is essential for AGI but dangerous when unbounded. PRE enforces:

- fixed recursion depth limits,
- evaluator-driven termination,
- disagreement-triggered aborts,
- deterministic control flow.

These constraints ensure that reasoning cannot spiral into unsafe or unbounded trajectories. Every recursive step is explicit, logged, and auditable.

15.4 Plural Evaluators and Epistemic Diversity

CAS-N introduces multiple evaluators with independent error geometries. This provides:

- **epistemic redundancy**: no single evaluator can dominate,
- **uncertainty detection**: disagreement signals instability,
- **automatic aborts**: unsafe states halt the reasoning process,
- **robustness**: evaluators catch each other’s blind spots.

Plurality is a structural safety mechanism: it prevents attractor collapse and ensures that unsafe or incoherent candidates are rejected early.

15.5 Deterministic Arbitration

Arbitration is deterministic and transparent. It does not rely on:

- stochastic sampling,
- neural variability,
- implicit preferences encoded in weights.

Instead, arbitration uses:

- explicit CAS vectors,
- weighted aggregation,
- consensus rules,
- policy constraints.

This ensures that decision-making is reproducible and auditable.

15.6 Script Engine Execution Guarantees

Generated cognitive programs are executed by a deterministic script engine. This provides:

- **reproducibility**: identical inputs yield identical outputs,
- **auditability**: every step is logged,
- **constraint enforcement**: only domain-valid actions can be executed,
- **sandboxing**: execution cannot escape domain boundaries.

The script engine acts as a final safety layer, ensuring that even if reasoning produces an unsafe candidate, it cannot be executed.

15.7 Governance Through Observability

Cloud-native deployment provides built-in observability:

- structured logs for every reasoning step,
- metrics for recursion depth and evaluator disagreement,
- traces for end-to-end workflows,
- policy layers for domain access and recursion limits.

Governance is therefore not an external process but an architectural property.

15.8 Comparison with Monolithic LLM Safety

Monolithic LLMs cannot provide structural safety because:

- their internal representations are opaque,
- their action space is unbounded,
- their reasoning is implicit,
- their behaviour is non-deterministic,
- their safety depends on behavioural heuristics.

Paged-PRE-CAS avoids these limitations entirely by:

- externalizing domain logic,
- enforcing domain boundaries,
- using explicit reasoning loops,
- separating evaluation dimensions,
- ensuring deterministic execution.

15.9 Failure Modes and Mitigations

The architecture mitigates common AGI failure modes:

Hallucination Prevented by domain constraints and evaluator checks.

Runaway recursion Prevented by bounded recursion and evaluator-driven termination.

Unsafe generalization Prevented by domain isolation and explicit action vocabularies.

Mode collapse Prevented by evaluator plurality and ensemble reasoning.

Opaque decision-making Prevented by explicit reasoning traces and deterministic arbitration.

15.10 Summary of Safety Guarantees

Paged-PRE-CAS provides the following structural guarantees:

- domain isolation,
- bounded recursion,
- evaluator plurality,
- deterministic arbitration,
- explicit reasoning,
- transparent state transitions,
- deterministic execution,
- full observability and governance.

These guarantees make the architecture inherently safe, transparent, and governable—properties that cannot be achieved through monolithic scaling or agent-based orchestration.

16 Discussion

The architecture presented in this paper proposes a structural alternative to monolithic language-model-based AI systems. Rather than relying on emergent reasoning within a single parameter space, it decomposes cognition into explicit, modular, and governable components. This section reflects on the broader implications of the Paged-PRE-CAS framework, its relationship to existing paradigms, and its potential impact on the development of safe and scalable AGI.

16.1 A Shift from Parametric to Structural Intelligence

Most contemporary AI systems derive their capabilities from parametric learning: knowledge and reasoning strategies are encoded implicitly in neural weights. This approach has produced remarkable results but suffers from fundamental limitations, including opacity, entanglement, and lack of governance.

Paged-PRE-CAS represents a shift toward *structural intelligence*, where:

- reasoning is explicit rather than emergent,
- evaluation is orthogonal and plural rather than entangled,
- domain logic is externalized rather than embedded in weights,

- safety is architectural rather than behavioural.

This shift parallels the evolution of software engineering from monolithic programs to microservice architectures: modularity, observability, and replaceability become first-class properties.

16.2 Implications for AGI Safety

The architecture demonstrates that AGI safety does not require solving alignment in the traditional sense. Instead, safety can emerge from:

- domain isolation,
- bounded recursion,
- evaluator plurality,
- deterministic arbitration,
- explicit reasoning traces,
- cloud-native observability.

These mechanisms provide structural guarantees that are impossible to achieve in monolithic LLMs or agent-based systems. The architecture therefore reframes AGI safety as a problem of *system design* rather than *behavioural alignment*.

16.3 Implications for AGI Capability

Although the architecture is safety-first, it also provides a path toward scalable capability:

- domain-bound AGIs can be added incrementally,
- search engines provide deep reasoning without parametric scaling,
- ensembles of reasoning cores provide robustness and diversity,
- mediator-LLMs enable natural language interaction without entangling cognition.

Capability emerges from the composition of many small intelligences rather than from scaling a single model. This approach is inherently extensible and avoids the diminishing returns of monolithic scaling.

16.4 Novelty of the Architecture

The architecture introduces several novel concepts:

- **Paged cognition:** domain decomposition of the action space.
- **PRE:** a minimal, explicit recursive reasoning kernel.
- **CAS:** orthogonal epistemic evaluation vectors.
- **CAS-N:** plural evaluators with disagreement-based aborts.
- **Domain-bound AGIs:** atomic cognitive units with explicit interfaces.
- **Generative reasoning:** generation of cognitive programs rather than tokens.
- **Cloud-native cognition:** AGI as a distributed microservice fabric.

These components combine into a coherent architecture that differs fundamentally from both LLM-based systems and classical cognitive architectures.

16.5 Relationship to Existing Paradigms

The architecture bridges several traditions:

- from cognitive architectures: explicit structure and modularity,
- from symbolic AI: deterministic reasoning and search,
- from modern AI: neural semantic interfaces,
- from distributed systems: microservices and observability,
- from safety engineering: structural constraints and governance.

However, it is not reducible to any of these paradigms. Its novelty lies in the integration of these ideas into a unified, safety-first AGI architecture.

16.6 Broader Implications

If adopted, this architecture could reshape how AGI systems are built:

- AGI development becomes an engineering discipline rather than a scaling race.
- Safety becomes a property of system design rather than behavioural tuning.
- Reasoning becomes explicit, inspectable, and auditable.
- Domain expertise becomes modular and replaceable.
- LLMs become optional semantic engines rather than cognitive cores.

This reframing has implications for research, industry, governance, and public understanding of AGI.

17 Limitations and Open Questions

Although the Paged-PRE-CAS architecture provides a structurally safe and modular foundation for AGI, several limitations and open questions remain. These limitations are not flaws of the architecture but areas where further engineering, empirical validation, or theoretical refinement is required. This section outlines the most important challenges and opportunities for future research.

17.1 Engineering Complexity and System Integration

The architecture decomposes cognition into many interacting components: reasoning cores, domain-bound AGIs, evaluators, search engines, memory services, and mediator-LLMs. While this modularity is essential for safety and governance, it introduces engineering complexity.

Open questions include:

- How should large-scale deployments orchestrate thousands of reasoning cores?
- What are the optimal communication protocols between cognitive microservices?
- How can latency be minimized without compromising observability?

These questions are primarily engineering challenges rather than conceptual limitations.

17.2 Scaling Domain Libraries

The architecture assumes a library of domain-bound AGIs. While each domain module is small and replaceable, building and maintaining a large library raises practical questions:

- How many domains are required for broad real-world capability?
- What is the optimal granularity of domain decomposition?
- How should domain modules be versioned, validated, and governed?

The architecture supports optional self-expansion, but governance and validation remain open research areas.

17.3 Evaluator Design and Epistemic Geometry

CAS and CAS-N rely on evaluators with orthogonal error geometries. Designing such evaluators is non-trivial:

- How should epistemic dimensions be selected for new domains?
- How can evaluator independence be measured or guaranteed?
- What disagreement thresholds should trigger aborts?

These questions require empirical study across diverse domains.

17.4 Search Engine Selection and Optimization

Domain-bound AGIs rely on externalized search engines. While this provides transparency and determinism, it raises questions about:

- search engine selection for complex or hybrid domains,
- heuristic design and optimization,
- balancing search depth with computational cost.

Search is a powerful tool, but its performance depends on domain-specific heuristics that may require expert tuning.

17.5 Mediator-LLM Reliability

Although the mediator-LLM is not a cognitive core, it remains a potential source of error:

- misclassification of domains,
- incorrect translation of natural language into structured states,
- ambiguity in multi-domain tasks.

These issues can be mitigated through validation layers, but they remain open engineering questions.

17.6 Emergent Behaviour in Large-Scale Systems

Even though the architecture is designed to minimize emergent behaviour, large-scale compositions of domain-bound AGIs may exhibit unexpected interactions:

- How do many domain modules interact in long-horizon tasks?
- Can emergent meta-strategies arise across domains?
- How should cross-domain workflows be governed at scale?

These questions require empirical study in real-world deployments.

17.7 Human–AI Interaction and Usability

The architecture provides structural safety, but usability and human–AI interaction remain open questions:

- How should cognitive programs be visualized for human oversight?
- What interfaces best support debugging and governance?
- How can users understand and trust explicit reasoning traces?

These questions lie at the intersection of AI, HCI, and cognitive ergonomics.

17.8 Theoretical Foundations

Finally, several theoretical questions remain open:

- What is the minimal set of epistemic dimensions required for general reasoning?
- How does PRE recursion relate to classical models of deliberation and planning?
- Can domain-bound AGIs be formally composed into a unified cognitive theory?

These questions invite further research into the theoretical underpinnings of structural intelligence.

18 Testable Predictions

A scientific architecture must make claims that can be empirically tested and potentially falsified. Although Paged-PRE-CAS is primarily a structural and engineering framework, it nonetheless yields a number of concrete predictions about system behaviour, performance, safety, and epistemic dynamics. These predictions can be evaluated through controlled experiments across diverse domains and implementations.

18.1 Prediction 1: CAS-N Disagreement Correlates with Epistemic Uncertainty

The architecture predicts that evaluator disagreement is a reliable indicator of uncertainty:

- high disagreement should correlate with ambiguous or unsafe states,
- low disagreement should correlate with stable, high-confidence reasoning,
- disagreement spikes should precede reasoning failures or invalid candidates.

This prediction can be tested by measuring evaluator variance across tasks with known difficulty or ambiguity.

18.2 Prediction 2: Domain Isolation Eliminates Cross-Domain Error Propagation

Paging predicts that strict domain boundaries prevent cross-domain interference:

- errors in one domain should not affect reasoning in another,
- invalid cross-domain actions should never be generated,
- domain-specific evaluators should reject mixed-domain states with near-perfect accuracy.

This can be tested by intentionally injecting cross-domain perturbations and measuring system resilience.

18.3 Prediction 3: PRE Recursion Depth Exhibits Diminishing Returns

The architecture predicts that increasing PRE recursion depth improves solution quality only up to a shallow threshold. Empirical observations in both discrete domains and token-generation models suggest:

- $k = 1$ or $k = 2$ is sufficient for most tasks,
- $k = 4$ can provide additional refinement in more complex cases,
- deeper recursion yields rapidly diminishing returns,
- excessive depth increases computational cost without improving quality.

This prediction can be tested by varying recursion depth across planning, optimization, and generation tasks and measuring improvements in evaluator scores, solution optimality, and consistency.

18.4 Prediction 4: Externalized Search Outperforms LLM-Simulated Search

Because domain-bound AGIs use algorithmic search rather than linguistic simulation, the architecture predicts:

- search-based reasoning should outperform LLM-generated trajectories,
- search should produce consistent and reproducible results,
- LLM-based search should show higher variance and more hallucinated states.

This can be tested by comparing search engines with LLM-generated plans in identical domains.

18.5 Prediction 5: Arbitration Controls, Rather Than Eliminates, Variance

The architecture predicts that arbitration provides explicit control over output variance. In its default deterministic mode, arbitration:

- reduces variance across repeated runs,
- improves reproducibility,
- eliminates stochastic drift in long-horizon tasks.

However, when a near-optimal threshold is enabled (e.g., selecting any candidate within 98% of the top candidate score), the arbiter introduces *controlled variation*. This yields:

- increased diversity of cognitive programs,
- bounded and predictable variance,
- no hallucinated or unsafe deviations, since all candidates remain domain-valid.

This prediction can be tested by comparing deterministic arbitration with threshold-based arbitration and measuring variance, diversity, and safety across repeated runs.

18.6 Prediction 6: Ensembles of Reasoning Cores Improve Robustness

Plural reasoning predicts that ensembles:

- reduce failure rates,
- improve solution quality,
- provide early warning signals through divergence across cores.

This can be tested by comparing single-core and multi-core configurations.

18.7 Prediction 7: Script Engine Execution Eliminates Hallucination

Because cognitive programs are executed by a deterministic script engine, the architecture predicts:

- hallucinated actions should be structurally impossible,
- invalid actions should be rejected before execution,
- execution traces should be fully reproducible.

This can be tested by attempting to induce hallucinated or invalid actions during execution.

18.8 Prediction 8: Mediator-LLM Errors Are Contained and Non-Catastrophic

Because the mediator-LLM is not a cognitive core, the architecture predicts:

- mediator errors should not propagate into reasoning,
- domain misclassification should be detectable by domain validators,
- incorrect translations should be caught by evaluators or domain constraints.

This can be tested by injecting controlled translation errors.

18.9 Prediction 9: Domain-Bound AGIs Scale Linearly with Domain Count

The architecture predicts that capability scales with the number of domain modules:

- adding new domains should increase capability without retraining,
- performance should degrade gracefully when domains are removed,
- scaling should be linear rather than exponential.

This can be tested by incrementally expanding or contracting domain libraries.

18.10 Prediction 10: Observability Enables Predictable Failure Modes

Because all reasoning steps are logged and observable, the architecture predicts:

- failure modes should be predictable and diagnosable,
- evaluator disagreement should precede failure,
- recursion-depth saturation should correlate with task difficulty.

This can be tested by analyzing logs across large-scale deployments.

19 Conclusion

This paper introduced Paged-PRE-CAS, a modular and structurally safe architecture for domain-bound AGI. Unlike monolithic language models or agent-based systems, the architecture does not rely on emergent reasoning, implicit representations, or unbounded action spaces. Instead, it decomposes cognition into explicit, inspectable, and governable components: domain-bound AGIs, PRE-based recursive reasoning, CAS-based epistemic evaluation, plural evaluators, deterministic arbitration, and cloud-native orchestration.

The central contribution of the architecture is the shift from *parametric intelligence* to *structural intelligence*. Reasoning is implemented as an explicit computational process, evaluation is orthogonal and plural, and domain logic is externalized into modular interfaces. Safety emerges from architectural constraints—domain isolation, bounded recursion, evaluator plurality, deterministic execution—rather than behavioural heuristics or alignment patches.

The architecture also demonstrates that AGI capability can scale without monolithic models. Domain-bound AGIs provide atomic cognitive units that can be added, replaced, or upgraded independently. Search engines provide deep reasoning without parametric scaling. Mediator-LLMs enable natural language interaction without entangling cognition. Cloud-native deployment provides observability, reproducibility, and governance.

Several limitations and open questions remain, particularly in evaluator design, domain library scaling, and large-scale orchestration. However, these challenges are engineering problems rather than conceptual obstacles. The architecture is implementable today, as demonstrated by the accompanying discrete state-transition domains and reasoning engines.

Ultimately, Paged-PRE-CAS reframes AGI development as a systems engineering discipline. Cognition becomes modular, explicit, and inspectable. Safety becomes structural. Capability becomes compositional. And general intelligence emerges not from scaling a single model but from the coordinated interaction of many small, coherent intelligences.

The architecture therefore offers a practical, extensible, and governable path toward safe AGI—one that is compatible with current technology, grounded in explicit computation, and aligned with the principles of transparency, modularity, and structural safety.

A PRE-CAS Overview Table

Architectural Insight

PRE-CAS is not a monolithic intelligence. It is a composition of minds: independent experts, evaluators, and arbiters whose coordinated interaction produces coherent reasoning.

Component	Input	Output	Role	Guarantees	Variations
Paging	Context c	Page index p	Selects domain for next action	Deterministic routing; domain coherence	Rule-based, learned, hybrid
Expert	c, a_{prev}	Proposal logits / action	Generates actions within one domain	No cross-domain errors	Single or multiple experts per page
PRE-Propose	$(c, a_{\text{prev}}, \emptyset, \emptyset)$	Initial proposal a_0	Starts refinement loop	Traceable, domain-bounded proposal	Neural, symbolic, heuristic
PRE-Refine	$(c, a_{\text{prev}}, a_j, e_j)$	Refined a_{j+1}	Improves proposal using evaluator feedback	Corrects weak proposals	Any refinement depth k
Evaluators	Action a , context c	Score vector e	Assess orthogonal constraints	Independent signals; full traceability	Heuristic, learned, hybrid
Filtering	Candidates (a_j, e_j)	Filtered set F	Removes invalid / low-quality actions	Hard constraints enforced	Threshold tuning; fallback rules
Arbiter	Filtered set F	Final action a_{final}	Selects best action with variation control	Transparent scoring; bounded diversity	Score clusters; sampling rules
Ensemble	Context c	$\{a_1, \dots, a_M\}$	Parallel independent reasoning	Plurality; robustness; ecology	With/without global arbiter

Table 1: Concise overview of the PRE-CAS modular reasoning architecture.

B Empirical Demonstration of Paged-PRE-CAS as a Generative Reasoning Model

This appendix presents a minimal generative micro-domain used to empirically validate the Paged-PRE-CAS architecture. The purpose is not to introduce a new model or dataset, but to demonstrate—under fully transparent and controlled conditions—that:

1. Paged-PRE-CAS functions as a generative reasoning model,
2. each architectural component contributes measurably to correctness and stability,
3. coherent, domain-valid sequences emerge from explicit reasoning rather than statistical sampling.

The micro-domain is intentionally small, interpretable, and structurally constrained. It provides a clean environment in which the behaviour of paging, PRE recursion, evaluator guidance, filtering, and arbitration can be observed in isolation.

B.1 Micro-Domain Setup

The generative domain consists of:

- a finite vocabulary partitioned into non-overlapping *pages*,
- a fixed-position context window,
- lightweight input and output embeddings,
- a set of orthogonal evaluators,
- a deterministic script engine that executes generated sequences.

Each token belongs to exactly one page. Paging ensures that only domain-valid tokens can be proposed at each step, enforcing structural safety by design.

B.2 Generative Reasoning Loop

Generation proceeds through the full Paged-PRE-CAS loop:

1. **Paging** selects the active domain page.
2. **Propose**: the page-expert produces an initial candidate token.
3. **Evaluate**: all evaluators score the candidate along orthogonal dimensions.
4. **Refine**: the expert receives evaluator feedback and proposes improved alternatives.
5. **Filter**: invalid candidates are removed before selection.
6. **Arbitrate**: the arbiter selects the final token from the filtered candidate set.
7. **Execute**: the token is appended to the sequence by the script engine.

A typical generation trace contains multiple refinement steps, several candidate proposals per step, evaluator scores, filtering decisions, and a final arbitration decision. All intermediate states are logged and fully inspectable.

C Ablation Studies

To validate the contribution of each architectural component, we perform controlled ablations within the micro-domain. All experiments use the same vocabulary, paging structure, evaluators, and training procedure unless otherwise noted.

C.1 Effect of Refinement

We compare refinement depths $k = 0, 1, 2$.

- $k = 0$: highly unstable behaviour; very low diversity; the arbiter has too few alternatives.
- $k = 1$: correctness and diversity increase substantially; refinement corrects weak proposals.
- $k = 2$: further improvement with diminishing returns; refinement stabilizes reasoning.

Conclusion: refinement is essential for stability and correctness.

C.2 Effect of Evaluators

Evaluators are disabled individually to measure their contribution.

C.2.1 Coherence Evaluator Disabled

Correctness decreases, diversity increases sharply, and semantic drift appears. The coherence evaluator provides local semantic continuity.

C.2.2 Style Evaluator Disabled

Correctness remains high, but stylistic consistency disappears and diversity increases. The style evaluator enforces CAS-guided stylistic alignment.

C.2.3 Heuristic Structural Evaluators Disabled

Disabling actor, timing, or line-structure evaluators causes correctness to collapse. Many sequences become structurally invalid and are filtered out.

Conclusion: evaluators are orthogonal and essential; each governs a distinct dimension of correctness.

C.3 Effect of Arbiter Threshold

The arbiter selects among high-scoring candidates using a score threshold.

- **High threshold (e.g., 0.98):** correctness high, diversity low, narrow attractors.
- **Lower threshold (e.g., 0.60):** correctness decreases, diversity increases, broader manifold.

Conclusion: the threshold provides a governable diversity dial; variation is controlled, bounded, and structurally safe.

D Summary of Findings

Across all experiments, the micro-domain demonstrates that:

- Paged-PRE-CAS generates coherent sequences through explicit reasoning,
- paging enforces domain isolation and prevents structural errors,
- refinement stabilizes generation and improves correctness,
- evaluators provide orthogonal epistemic signals,
- filtering before selection ensures safety,
- arbitration balances correctness with controlled diversity,
- ensembles provide robustness and plural reasoning trajectories.

The system produces valid sequences with fully traceable reasoning steps, confirming that generative behaviour emerges from structured recursion, evaluator guidance, and domain isolation.

E Implications for the Main Architecture

The micro-domain experiments validate the core claims of the Paged-PRE-CAS architecture:

1. generative reasoning is feasible without monolithic models or token sampling,
2. explicit structure—paging, PRE recursion, CAS evaluation—produces stable, coherent generative behaviour,
3. safety mechanisms (domain isolation, filtering, evaluator plurality) operate as intended,
4. the architecture is modular, inspectable, and fully traceable.

These results support the use of Paged-PRE-CAS as a safe, modular, and transparent generative reasoning engine within the broader AGI architecture.

Companion Work and Ecosystem Context

This paper is part of the open Triadic Cosmos ecosystem, which develops a unified conceptual and computational framework for understanding both physical structure and the emergence of intelligent systems. Its companion work, *The Triadic Cycle as Unified Framework*, presents the foundational physical ontology based on the roles of information, energy, and geometry. While the present paper does not derive its AGI architecture directly from the Triadic Cycle, both works share a common motivation: extracting minimal structural principles from complex systems.

The AGI architecture developed here is grounded in a distinct structural model—the Quad—which captures the functional organization of intelligent behaviour. The Triadic Cycle provides the broader ontological backdrop of the ecosystem, whereas the Quad provides the operational architecture for scalable intelligence.

All materials, extended notes, and evolving documentation are available in the open-source repository: <https://github.com/triadic-cosmos>.

References

- Amodei, Dario et al. (2016). “Concrete Problems in AI Safety”. In: *arXiv:1606.06565*.
- Anderson, John R. (2007). *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press.
- Authors, The Kubernetes (2023). *Kubernetes Documentation*. <https://kubernetes.io>.
- Baars, Bernard J. (1988). “A Cognitive Theory of Consciousness”. In: *Cambridge University Press*.
- Chase, Harrison (2023). *LangChain*. <https://github.com/langchain-ai/langchain>.
- Christiano, Paul et al. (2017). “Deep Reinforcement Learning from Human Preferences”. In: *NeurIPS*.
- Dean, Jeffrey and Sanjay Ghemawat (2004). “MapReduce: Simplified Data Processing on Large Clusters”. In: *OSDI*.
- Dehaene, Stanislas, Michel Kerszberg, and Jean-Pierre Changeux (1998). “A Neuronal Model of a Global Workspace in Effortful Cognitive Tasks”. In: *Proceedings of the National Academy of Sciences* 95.24, pp. 14529–14534.
- Fedus, William, Barret Zoph, and Noam Shazeer (2022). “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *JMLR*.
- Hendrycks, Dan et al. (2022). “X-Risk Analysis for AI Research”. In: *arXiv:2206.05862*.
- Hutchins, Edwin (1995). *Cognition in the Wild*. MIT Press.
- Laird, John E. (2012). *The Soar Cognitive Architecture*. MIT Press.
- Lepikhin, Dmitry et al. (2021). “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding”. In: *ICLR*.
- Microsoft (2023). *Semantic Kernel*. <https://github.com/microsoft/semantic-kernel>.
- Minsky, Marvin (1986). *The Society of Mind*. Simon and Schuster.
- Moritz, Philipp et al. (2018). “Ray: A Distributed Framework for Emerging AI Applications”. In: *OSDI*.
- Nanda, Neel et al. (2023). *A Mechanistic Interpretability Analysis of Grokking*. *arXiv:2301.05217*.
- Newman, Sam (2015). *Building Microservices*. O’Reilly Media.
- Olah, Chris et al. (2020). “Zoom In: An Introduction to Circuits”. In: *Distill*.
- Ouyang, Long et al. (2022). “Training Language Models to Follow Instructions with Human Feedback”. In: *arXiv:2203.02155*.
- Shazeer, Noam et al. (2017). “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *ICLR*.
- Sun, Ron (2016). *The CLARION Cognitive Architecture: Toward a Comprehensive Theory of the Mind*. Oxford University Press.
- Woolley, Anita Williams et al. (2010). “Evidence for a Collective Intelligence Factor in the Performance of Human Groups”. In: *Science* 330.6004, pp. 686–688.
- Wu, Yizhou et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. <https://github.com/microsoft/autogen>.